

Глава 8

Детальное описание контейнеров STL

В главе подробно обсуждаются все операции, предусмотренные в контейнерах STL. Классы и их члены сгруппированы по функциональному признаку. Для каждого типа и операции в главе приведены их сигнатуры, поведение и контейнерные типы, которые их предоставляют. Возможными контейнерными типами являются массив, вектор, дек, список, последовательный список, множество, мультимножество, отображение, мультиотображение, неупорядоченное множество, неупорядоченное мультимножество, неупорядоченное отображение, неупорядоченное мультиотображение и строка. В следующих разделах слово “контейнер” означает контейнерный тип, содержащий соответствующий член.

8.1. Определения типов

контейнер : **value_type**

- Тип элементов.
- Для (неупорядоченных) множеств и мультимножеств является константным.
- Для (неупорядоченных) отображений и мультиотображений является классом `pair <const тип-ключа, отображаемый-тип>`.
- Предусмотрен в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

контейнер : **reference**

- Тип ссылок на элементы.
- Как правило, *контейнер* : `value_type&`.
- В классе `vector<bool>` является вспомогательным классом (см. раздел 7.3.6).
- Предусмотрен в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

контейнер : **const_reference**

- Тип ссылок на элементы только для чтения.
- Как правило, `const контейнер : value_type&`.

- В классе `vector<bool>` является типом `bool`.
- Предусмотрен в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

контейнер : **iterator**

- Тип итераторов.
- Предусмотрен в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

контейнер : **const_iterator**

- Тип итераторов только для чтения.
- Предусмотрен в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

контейнер : **reverse_iterator**

- Тип обратных итераторов.
- Предусмотрен в массиве, векторе, деке, списке, множестве, мультимножестве, отображении, мультиотображении и строке.
- *контейнер* : **const_reverse_iterator**
- Тип обратных итераторов только для чтения.
- Предусмотрен в массиве, векторе, деке, списке, множестве, мультимножестве, отображении, мультиотображении и строке.

контейнер : **pointer**

- Тип указателей на элементы.
- Предусмотрен в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

контейнер : **const_pointer**

- Тип указателей на элементы только для чтения.
- Предусмотрен в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

контейнер : **size_type**

- Целочисленный тип для размера без знака.
- Предусмотрен в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном

множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

контейнер : **difference_type**

- Целочисленный тип для размера со знаком.
- Предусмотрен в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

контейнер : **key_type**

- Тип ключей элементов ассоциативных и неупорядоченных контейнеров.
- В (неупорядоченных) множествах и мультимножествах эквивалентен типу `value_type`.
- Предусмотрен в множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении.

контейнер : **mapped_type**

- Тип значения элементов ассоциативных и неупорядоченных контейнеров.
- Предусмотрен в отображении, мультиотображении, неупорядоченном отображении, неупорядоченном мультиотображении.

контейнер : **key_compare**

- Тип критерия сравнения в ассоциативном контейнере.
- Предусмотрен в множестве, мультимножестве, отображении и мультиотображении.

контейнер : **value_compare**

- Тип критерия сравнения для типа элементов.
- В множествах и мультимножествах эквивалентен `key_compare`.
- В отображениях и мультиотображениях является вспомогательным классом для критерия сравнения, предназначенного для сравнения ключей двух элементов.
- Предусмотрен в множестве, мультимножестве, отображении и мультиотображении.

контейнер : **hasher**

- Тип функции хеширования в неупорядоченных контейнерах.
- Предусмотрен в неупорядоченном множествах, неупорядоченном мультимножестве, неупорядоченном отображении и неупорядоченном мультиотображении.

контейнер : **key_equal**

- Тип предиката эквивалентности в неупорядоченных контейнерах.
- Предусмотрен в неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении и неупорядоченном мультиотображении.

контейнер: **:local_iterator**

- Тип сегментных итераторов в неупорядоченных контейнерах.
- Доступен, начиная со стандарта C++11.
- Предусмотрен в неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении и неупорядоченном мультиотображении.

контейнер: **:const_local_iterator**

- Тип сегментных итераторов только для чтения в неупорядоченных контейнерах.
- Доступен, начиная со стандарта C++11.
- Предусмотрен в неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении и неупорядоченном мультиотображении.

Кроме того, тип `allocator_type` предусмотрен для всех контейнеров, за исключением массивов (см. раздел 8.10.1).

8.2. Операции создания, копирования и удаления

Контейнеры имеют конструкторы и деструкторы, перечисленные ниже. Кроме того, большинство конструкторов позволяют передавать модель памяти в качестве дополнительного аргумента, указанного в разделе 8.10. Обсуждение вопросов, связанных с инициализацией контейнеров, приведено в разделе 7.1.2.

контейнер: **:контейнер ()**

- Конструктор по умолчанию.
- Создает новый пустой контейнер.
- В массивах эта операция определена неявно и создает непустой контейнер, в котором элементы могут иметь неопределенные значения (см. раздел 7.2.1).
- Предусмотрен в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

`explicit контейнер::контейнер (const CompFunc& cmpPred)`

- Создает новый пустой контейнер с предикатом *cmpPred* в качестве критерия сортировки (см. примеры в разделах 7.7.5 и 7.8.6).
- Критерий сортировки должен определять *строгое слабое упорядочение* (см. раздел 7.7).
- Предусмотрен в множестве, мультимножестве, отображении и мультиотображении.

`explicit контейнер::контейнер (size_type bnum)`

`explicit контейнер::контейнер (size_type bnum, const Hasher& hasher)`

`explicit контейнер::контейнер (size_type bnum, const Hasher& hasher, const KeyEqual& eqPred)`

- Создают новый пустой контейнер, имеющий не менее *bnum* сегментов, функцию хеширования *hasher* и критерий эквивалентности *eqPred*.

- Если критерий эквивалентности *eqPred* не передается, используется критерий эквивалентности, предусмотренный по умолчанию для контейнерного типа (см. раздел 7.9.2).
- Если функция хеширования *hasher* не передается, используется функция хеширования, предусмотренная по умолчанию для контейнерного типа (см. раздел 7.9.2).
- Предусмотрены в неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении и неупорядоченном мультитообразии.

контейнер : : *контейнер* (*список-инициализации*)

- Создает новый контейнер, инициализированный элементами *списка инициализации*.
- Для массивов эта операция определена неявно (см. раздел 7.2.1).
- Доступен начиная со стандарта C++11.
- Предусмотрен в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультитообразии, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультитообразии, строке.

контейнер : : *контейнер* (*список-инициализации*, *const CompFunc& cmpPred*)

- Создает контейнер, имеющий критерий сортировки *cmpPred* и инициализированный *списком инициализации*.
- Критерий сортировки должен определять *строгое слабое упорядочение* (см. раздел 7.7).
- Доступен начиная со стандарта C++11.
- Предусмотрен в множестве, мультимножестве, отображении и мультитообразии.

контейнер : : *контейнер* (*список-инициализации*, *size_type bnum*)

контейнер : : *контейнер* (*список-инициализации*, *size_type bnum*,
const Hasher& hasher)

контейнер : : *контейнер* (*список-инициализации*, *size_type bnum*,
const Hasher& hasher, *const KeyEqual& eqPred*)

- Создает контейнер, имеющий не менее *bnum* сегментов, функцию хеширования *hasher*, критерий эквивалентности *eqPred* и инициализированный элементами *списка инициализации*.
- Если критерий эквивалентности *eqPred* не передается, используется критерий эквивалентности, предусмотренный для контейнерного типа по умолчанию (см. раздел 7.9.2).
- Если функция хеширования *hasher* не передается, используется функция хеширования, предусмотренная для контейнерного типа по умолчанию (см. раздел 7.9.2).
- Предусмотрены в неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении и неупорядоченном мультитообразии.

контейнер : : *контейнер* (*const контейнер& c*)

- Копирующий конструктор.
- Создает новый контейнер как копию существующего контейнера *c*.

- Вызывает копирующий конструктор каждого элемента контейнера *c*.
- Для массивов эта операция определена неявно.
- Предусмотрен в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

контейнер::контейнер (*контейнер&& c*)

- Перемещающий конструктор.
- Создает новый контейнер, инициализированный элементами существующего контейнера *c*.
- После вызова перемещающего конструктора контейнер *c* остается корректным, но имеет неопределенное значение.
- Для массивов эта операция определена неявно.
- Доступна, начиная со стандарта C++11.
- Предусмотрен в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

explicit контейнер::контейнер (*size_type num*)

- Создает контейнер, состоящий из *num* элементов.
- Элементы создаются своими конструкторами, заданными по умолчанию.
- Предусмотрен в векторе, деке, списке и последовательном списке.

контейнер::контейнер (*size_type num, const T& value*)

- Создает контейнер, состоящий из *num* элементов.
- Элементы создаются как копии аргумента *value*.
- *T* — тип элементов контейнера.
- Для строк аргумент *value* не передается по ссылке.
- Предусмотрен в векторе, деке, списке, последовательном списке и строке.

контейнер::контейнер (*InputIterator beg, InputIterator end*)

- Создает контейнер, инициализированный всеми элементами диапазона [*beg, end*).
- Эта функция-член является шаблонной (см. раздел 3.2). Следовательно, элементы диапазона-источника могут иметь любой тип, который может быть преобразован в тип элементов контейнера.
- Примеры и обсуждения проблем, возникающих при работе с этой функцией-членом, см. в разделе 7.1.2.
- Предусмотрен в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

`контейнер::контейнер` (InputIterator *beg*, InputIterator *end*,
const CompFunc& *cmpPred*)

- Создает контейнер, имеющий критерий сортировки *cmpPred* и инициализированный всеми элементами диапазона [*beg*,*end*).
- Эта функция-член является шаблонной (см. раздел 3.2). Следовательно, элементы диапазона-источника могут иметь любой тип, который может быть преобразован в тип элементов контейнера.
- Критерий сортировки должен определять *строгое слабое упорядочение* (см. раздел 7.7).
- Предусмотрен в множестве, мультимножестве, отображении, мультиотображении.

`контейнер::контейнер` (InputIterator *beg*, InputIterator *end*,
size_type *bnum*)

`контейнер::контейнер` (InputIterator *beg*, InputIterator *end*,
size_type *bnum*, const Hasher& *hasher*)

`контейнер::контейнер` (InputIterator *beg*, InputIterator *end*,
size_type *bnum*, const Hasher& *hasher*, const KeyEqual& *eqPred*)

- Создает контейнер, имеющий не менее *bnum* сегментов, функцию хеширования *hasher*, критерий эквивалентности *eqPred* и инициализированный всеми элементами диапазона [*beg*,*end*).
- Если критерий эквивалентности *eqPred* не передается, используется критерий эквивалентности, предусмотренный по умолчанию для данного контейнерного типа (см. раздел 7.9.2).
- Если функция хеширования *hasher* не передается, используется функция хеширования, предусмотренная по умолчанию для данного контейнерного типа (см. раздел 7.9.2).
- Предусмотрены в неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении и неупорядоченном мультиотображении.

`контейнер::~~контейнер` ()

- Деструктор.
- Удаляет все элементы и освобождает память.
- Вызывает деструктор каждого элемента.
- Предусмотрен в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

8.3. Немодифицирующие операции

8.3.1. Операции над размером

`bool контейнер::empty` () const

- Возвращает признак того, что контейнер пуст (не содержит элементов).
- Эквивалент выражения `begin() == end()`, но может работать быстрее.

- Сложность: константная.
- Предусмотрен в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

`size_type контейнер::size () const`

- Возвращает текущее количество элементов.
- Для проверки, пуст ли контейнер (не содержит элементов), необходимо использовать функцию-член `empty()`, которая может работать быстрее.
- Сложность: константная.
- Предусмотрен в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

`size_type контейнер::max_size () const`

- Возвращает максимальное количество элементов, которые может содержать контейнер.
- Это техническое значение, которое может зависеть от модели памяти, используемой в контейнере. В частности, поскольку векторы всегда занимают один сегмент памяти, эта величина для вектора может оказаться меньше, чем для других контейнеров.
- Сложность: константная.
- Предусмотрен в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

8.3.2. Операции сравнения

`bool operator == (const контейнер& c1, const контейнер& c2)`

`bool operator != (const контейнер& c1, const контейнер& c2)`

- Возвращает признак того, что два контейнера (не) равны.
- Два контейнера считаются равными, если они содержат одинаковое количество элементов и состоят из одних и тех же элементов (т.е. для всех сравнений соответствующих элементов операция `==` возвращает значение `true`). За исключением неупорядоченных контейнеров, одинаковые элементы хранятся в одном и том же порядке.
- Сложность: как правило, линейная. Для неупорядоченных контейнеров в наихудшем случае является квадратичной.
- Предусмотрен в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

```
bool operator < (const контейнер& c1, const контейнер& c2)
bool operator <= (const контейнер& c1, const контейнер& c2)
bool operator > (const контейнер& c1, const контейнер& c2)
bool operator >= (const контейнер& c1, const контейнер& c2)
```

- Возвращают результат сравнения двух контейнеров одинакового типа.
- Для проверки того, является ли один контейнер меньше другого, контейнеры сравниваются в лексикографическом порядке (описание алгоритма `lexicographical_compare()` приведено в разделе 11.5.4).
- Сложность: линейная.
- Предусмотрен в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, строке.

8.3.3. Немодифицирующие операции над ассоциативными и неупорядоченными контейнерами

Функции-члены, перечисленные в этом разделе, представляют собой специальные реализации соответствующих алгоритмов STL, рассмотренных в разделах 11.5 и 11.9. Эти функции-члены обеспечивают более высокую производительность, потому что они используют тот факт, что элементы ассоциативных контейнеров упорядочиваются и фактически гарантируют логарифмическую, а не линейную сложность операций. Например, поиск элемента среди 1000 элементов в среднем требует не более 10 сравнений (см. раздел 2.2). Если используется хорошая хеш-функция, то функции-члены могут иметь даже константную сложность.

```
size_type контейнер::count (const T& value) const
```

- Возвращает количество элементов, эквивалентных аргументу *value*.
- Представляет собой специальную версию алгоритма `count()` из раздела 11.5.1.
- T — тип упорядоченных значений:
 - для множеств и мультимножеств является типом элементов;
 - для отображения и мультиотображений является типом ключей.
- Сложность: логарифмическая для ассоциативных массивов и константная для неупорядоченных контейнеров, снабженных хорошей хеш-функцией.
- Предусмотрен в множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении.

```
iterator контейнер::find (const T& value)
```

```
const_iterator контейнер::find (const T& value) const
```

- Возвращают позицию первого элемента, имеющего значение, эквивалентное аргументу *value*.
- Если элемент не найден, возвращают итератор `end()`.
- Представляют собой специальные версии алгоритма `find()`, рассмотренного в разделе 11.5.3.

- `T` — тип упорядоченных значений:
 - для множеств и мультимножеств является типом элементов;
 - для отображений и мультиотображений является типом ключей.
- Сложность: логарифмическая для ассоциативных массивов и константная для неупорядоченных контейнеров, снабженных хорошей хеш-функцией.
- Предусмотрен в множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении.

`iterator контейнер::lower_bound (const T& value)`

`const_iterator контейнер::lower_bound (const T& value) const`

- Возвращают первую позицию, в которую можно вставить копию аргумента `value` в соответствии с критерием сортировки.
- Если элемент не найден, возвращают итератор `end()`.
- Возвращаемое значение представляет собой позицию первого элемента, равного или большего аргумента `value`, в соответствии с критерием сортировки (возможно `end()`).
- Представляют собой специальные версии алгоритма `lower_bound()`, рассмотренного в разделе 11.10.1.
- `T` — тип упорядоченных значений:
 - для множеств и мультимножеств является типом элементов;
 - для отображения и мультиотображений является типом ключей.
- Сложность: логарифмическая.
- Предусмотрены в множестве, мультимножестве, отображении и мультиотображении.

`iterator контейнер::upper_bound (const T& value)`

`const_iterator контейнер::upper_bound (const T& value) const`

- Возвращают последнюю позицию, в которую можно вставить копию аргумента `value` в соответствии с критерием сортировки.
- Если элемент не найден, возвращают итератор `end()`.
- Возвращаемое значение представляет собой позицию первого элемента, большего аргумента `value`, в соответствии с критерием сортировки (возможно, `end()`).
- Представляют собой специальные версии алгоритма `upper_bound()`, рассмотренного в разделе 11.10.1.
- `T` — тип упорядоченных значений:
 - для множеств и мультимножеств является типом элементов;
 - для отображений и мультиотображений является типом ключей.
- Сложность: логарифмическая.
- Предусмотрены в множестве, мультимножестве, отображении и мультиотображении.

```
pair<iterator, iterator> контейнер::equal_range (const T& value)
pair<const_iterator, const_iterator>
контейнер::equal_range (const T& value) const
```

- Возвращают первую и последнюю позиции, в которые можно вставить копию аргумента *value*, в соответствии с критерием сортировки.
- Возвращаемое значение представляет собой диапазон элементов, равных аргументу *value*.
- Эквивалент вызова `make_pair(lower_bound(value), upper_bound(value))`.
- Представляют собой специальные версии алгоритма `equal_range()`, рассмотренного в разделе 11.10.1.
- *T* — тип упорядоченных значений:
 - для множеств и мультимножеств является типом элементов;
 - для отображений и мультиотображений является типом ключей.
- Сложность: логарифмическая для ассоциативных массивов и константная для неупорядоченных контейнеров, снабженных хорошей хеш-функцией.
- Предусмотрен в множестве, мультимножестве, отображении и мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении и неупорядоченном мультиотображении.

8.4. Присваивание

```
контейнер& контейнер::operator = (const контейнер& c)
```

- Копирующая операция присваивания.
- Присваивает все элементы контейнера *c*; т.е. заменяет существующие элементы копиями элементов контейнера *c*.
- Эта операция может вызывать операцию присваивания для всех элементов, которые должны быть заменены, копирующий конструктор для добавляемых элементов и деструктор для удаляемых элементов.
- Предусмотрена в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

```
контейнер& контейнер::operator = (контейнер&& c)
```

- Перемещающая операция присваивания.
- Перемещает все элементы контейнера *c* в контейнер `*this`; т.е. заменяет существующие элементы копиями элементов контейнера *c*.
- После ее выполнения контейнер *c* остается корректным, но имеет неопределенное значение.
- Доступен начиная со стандарта C++11.

- Предусмотрена в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

`контейнер& контейнер::operator = (список-инициализации)`

- Присваивает все элементы из *списка инициализации*, т.е. заменяет все существующие элементы копиями передаваемых элементов.
- Эта операция может вызывать операцию присваивания для всех элементов, которые должны быть заменены, копирующий конструктор для добавляемых элементов и деструктор для удаляемых элементов.
- Доступен начиная со стандарта C++11.
- Предусмотрена в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

`void контейнер::assign (список-инициализации)`

- Присваивает все элементы из *списка инициализации*, т.е. заменяет все существующие элементы копиями передаваемых элементов.
- Доступна, начиная со стандарта C++11.
- Предусмотрена в векторе, деке, списке, последовательном списке и строке.

`void массив::fill (const T& value)`

- Присваивает аргумент *value* всем элементам, т.е. заменяет все существующие элементы копиями аргумента *value*.
- Доступна начиная со стандарта C++11.
- Предусмотрена в массиве.

`void контейнер::assign (size_type num, const T& value)`

- Присваивает *num* вхождений аргумента *value*, т.е. заменяет все существующие элементы *num* копиями аргумента *value*.
- *T* должен быть типом элемента.
- Предусмотрена в векторе, деке, списке, последовательном списке и строке.

`void контейнер::assign (InputIterator beg, InputIterator end)`

- Присваивает все элементы диапазона *[beg,end)*, т.е. заменяет все существующие элементы копиями элементов диапазона *[beg,end)*.
- Эта функция-член является шаблонной (см. раздел 3.2). Следовательно, элементы диапазона-источника могут иметь любой тип, который может быть преобразован в тип элементов контейнера.
- Предусмотрена в векторе, деке, списке, последовательном списке и строке.


```
void контейнер::swap (контейнер& c)
void swap (контейнер& c1, контейнер& c2)
```

- Обменивает содержимое текущего контейнера и контейнера *c* или содержимое контейнеров *c1* и *c2* соответственно.
- Обе функции выполняют обмен
 - элементов контейнеров;
 - их критериев сортировки, предикатов эквивалентности и хеш-функций, если таковые существуют.
- Ссылки, указатели и итераторы, установленные на элементы, меняют свои контейнеры, потому что после обмена они ссылаются на прежние элементы.
- Массивы не могут просто обменять внутренние указатели. Следовательно, функция `swap()` имеет линейную сложность, а итераторы и ссылки после обмена относятся к тому же самому контейнеру, но с другими элементами.
- Для ассоциативных контейнеров функция может генерировать исключение, только если копирование и присваивание критерия сравнения могут генерировать исключение. Для неупорядоченных контейнеров функция может генерировать исключение, только если предикат эквивалентности или хеш-функция могут генерировать исключение. Для всех остальных контейнеров функция не может генерировать исключение.
- Сложность: как правило, константная. Для массивов сложность является линейной.
- Если присваиваемый объект далее не нужен, то из-за невысокой сложности функции `swap()` всегда следует отдавать предпочтение ей, а не копирующему присваиванию (см. раздел 7.1.2).
- Предусмотрена в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

8.5. Прямой доступ к элементам

```
reference контейнер::at (size_type idx)
const_reference контейнер::at (size_type idx) const
```

- Возвращают элемент с индексом *idx* (первый элемент имеет индекс 0).
- Передача некорректного индекса (меньшего 0 или равного `size()` или большего `size()`) вызывает генерацию исключения `out_of_range`.
- Возвращаемая ссылка может стать некорректной из-за последующих модификаций или перераспределения памяти.
- Если вы уверены, что индекс корректен, можно использовать операцию `[]`, которая работает быстрее.
- Предусмотрены в массиве, векторе, деке и строке.

```
T& отображение::at (const key_type& key)
const T& отображение::at (const key_type& key) const
```

- Возвращают значение, соответствующее ключу *key* в отображении.
- Если в отображении нет элемента, ключ которого совпадает с аргументом *key*, генерируют исключение `out_of_range`.
- Доступны начиная со стандарта C++11.
- Предусмотрены в отображении и неупорядоченном отображении.

```
reference контейнер::operator[] (size_type idx)
const_reference контейнер::operator[] (size_type idx) const
```

- Возвращают элемент с индексом *idx* (первый элемент имеет индекс 0).
- Передача неправильного индекса (меньшего 0 или равного `size()` или большего `size()`) приводит к неопределенному поведению. Следовательно, вызывающая сторона должна гарантировать, что индекс является корректным; в противном случае следует использовать функцию-член `at()`.
- Возвращаемая ссылка может стать некорректной из-за последующих модификаций или перераспределения памяти.
- Предусмотрены в массиве, векторе, деке и строке.

```
T& отображение::operator[] (const key_type& key)
T& отображение::operator[] (key_type&& key)
```

- Операции `[]` для ассоциативных массивов.
- Возвращают значение, соответствующее ключу *key* в отображении.
- Если в отображении нет элемента, ключ которого совпадает с аргументом *key*, эти операции автоматически *создают* новый элемент с данным ключом (копируемым или перемещаемым) и значением, инициализированным конструктором, предусмотренным по умолчанию для данного типа значения. Таким образом, индекс никогда не может быть некорректным (возможно только неправильное поведение). Подробнее см. в разделах 6.2.4 и 7.8.3.
- Во второй версии состояние аргумента *key* после выполнения операции становится неопределенным (эта форма реализует семантику перемещения для случая, когда ключа еще не существует).
- Первая версия эквивалентна выражению

```
(*((insert(make_pair(key, T()))).first)).second
```

- Вторая версия доступна начиная со стандарта C++11.
- Предусмотрены в отображении и мультиотображении.

```
reference контейнер::front ()
const_reference контейнер::front () const
```

- Обе версии возвращают первый элемент (элемент с индексом 0).
- Вызывающая сторона должна гарантировать, что контейнер содержит хотя бы один элемент (`size() > 0`); в противном случае последствия непредсказуемы.

- Для строк обе операции доступны начиная со стандарта C++11.
- Предусмотрены в массиве, векторе, деке, списке, последовательном списке и строке.

```
reference контейнер::back ()
const_reference контейнер::back () const
```

- Обе функции возвращают последний элемент (элемент с индексом `size() - 1`).
- Вызывающая сторона должна гарантировать, что контейнер содержит хотя бы один элемент (`size() > 0`); в противном случае последствия непредсказуемы.
- Для строк обе операции доступны начиная со стандарта C++11.
- Предусмотрены в массиве, векторе, деке, списке и строке.

```
T* контейнер::data ()
const T* контейнер::data () const
```

- Обе функции возвращают обычный массив в стиле языка C со всеми элементами (т.е. указатель на первый элемент массива).
- Обе функции предназначены для передачи элементов массива интерфейсам в стиле языка C.
- Для строк доступна только вторая версия.
- Для массивов и векторов доступны начиная со стандарта C++11.
- Предусмотрены в массиве, векторе и строке.

8.6. Операции генерации итераторов

Следующие функции-члены возвращают итераторы для обхода элементов контейнера. В табл. 8.1 приведены категории итераторов (см. раздел 9.2) в соответствии с разными типами контейнеров.

Таблица 8.1. Категории итераторов, предусмотренных для контейнерных типов

Контейнер	Категория итератора
Массив	Произвольного доступа
Вектор	Произвольного доступа
Дек	Произвольного доступа
Список	Двунаправленный
Последовательный список	Однонаправленный
Множество	Двунаправленный; элемент — константа
Мультимножество	Двунаправленный; элемент — константа
Отображение	Двунаправленный; ключ — константа

Окончание табл. 8.1

Контейнер	Категория итератора
Мультиотображение	Двунаправленный; ключ — константа
Неупорядоченное множество	Однонаправленный; элемент — константа
Неупорядоченное мультимножество	Однонаправленный; элемент — константа
Неупорядоченное отображение	Однонаправленный; ключ — константа
Неупорядоченное мультиотображение	Однонаправленный; ключ — константа
Строка	Произвольного доступа

```
iterator контейнер::begin ()
const_iterator контейнер::begin () const
const_iterator контейнер::cbegin () const
```

- Возвращает итератор, установленный в начало контейнера (позицию первого элемента).
- Если контейнер пуст, вызов функции эквивалентен вызовам `контейнер::end()` или `контейнер::cend()` соответственно.
- В неупорядоченных контейнерах есть функции-члены `begin()` и `cbegin()` с числовым аргументом для обеспечения сегментного интерфейса (см. раздел 8.9.3).
- Функция `cbegin()` доступна начиная со стандарта C++11.
- Предусмотрена в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

```
iterator контейнер::end ()
const_iterator контейнер::end () const
const_iterator контейнер::cend () const
```

- Возвращает итератор, установленный в конец контейнера (позицию после последнего элемента).
- Если контейнер пуст, вызов функции эквивалентен вызовам `контейнер::begin()` или `контейнер::cbegin()` соответственно.
- В неупорядоченных контейнерах есть функции-члены `begin()` и `cbegin()` с числовым аргументом для обеспечения сегментного интерфейса (см. раздел 8.9.3).
- Функция `cend()` доступна начиная со стандарта C++11.
- Предусмотрена в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

```
reverse_iterator контейнер::rbegin ()
const_reverse_iterator контейнер::rbegin () const
const_reverse_iterator контейнер::crbegin () const
```

- Возвращает обратный итератор, установленный в начало контейнера при обратном обходе (позицию последнего элемента).
- Если контейнер пуст, вызов функции эквивалентен вызовам *контейнер*::rend() или *контейнер*::crend() соответственно.
- Подробное описание обратных итераторов приведено в разделе 9.4.1.
- Функция crbegin() доступна начиная со стандарта C++11.
- Предусмотрена в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении и строке.

```
reverse_iterator контейнер::rend ()
const_reverse_iterator контейнер::rend () const
const_reverse_iterator контейнер::crend () const
```

- Возвращает обратный итератор, установленный в конец контейнера при обратном обходе (позицию перед первым элементом).
- Если контейнер пуст, вызов функции эквивалентен вызовам *контейнер*::rbegin() или *контейнер*::crbegin() соответственно.
- Подробное описание обратных итераторов приведено в разделе 9.4.1.
- Функция crend() доступна начиная со стандарта C++11.
- Предусмотрена в массиве, векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении и строке.

8.7. Вставка и удаление элементов

8.7.1. Вставка отдельных элементов

```
iterator контейнер::insert (const T& value)
iterator контейнер::insert (T&& value)
pair<iterator,bool> контейнер::insert (const T& value)
pair<iterator,bool> контейнер::insert (T&& value)
```

- Вставляют аргумент *value* в ассоциативный или неупорядоченный контейнер.
- Первая и третья версии копируют аргумент *value*.
- Вторая и четвертая версии перемещают аргумент *value* в контейнер, так что после выполнения операций значение аргумента *value* становится неопределенным.
- Контейнеры, допускающие дубликаты (неупорядоченные), мультимножества и мультиотображения, имеют первую и вторую сигнатуры. Они возвращают позицию нового элемента. Начиная со стандарта C++11 вновь вставленные элементы гарантированно размещаются после существующих эквивалентных значений.

- Контейнеры, не допускающие дубликатов (неупорядоченные), множества и отображения, имеют третью и четвертую сигнатуры. Если вставка элемента невозможна из-за того, что в контейнере уже существует элемент с таким же значением или ключом, они возвращают позицию существующего элемента и значение `false`. Если вставка возможна, они возвращают позицию нового элемента и значение `true`.
- `T` — тип элементов контейнера. Следовательно, для (неупорядоченных) отображений и мультиотображений он представляет собой пару “ключ–значение”.
- Для отображения, мультиотображения, неупорядоченного отображения и неупорядоченного мультиотображения соответствующая форма с семантикой перемещения является шаблонной (см. раздел 3.2). Следовательно, аргумент *value* может иметь любой тип, который можно преобразовать в тип значения (пару “ключ–значение”) контейнера. Это позволяет передавать две строки так, чтобы первая из них могла быть преобразована в константную строку (ключ).
- Если хеш-функции в неупорядоченных контейнерах не генерируют исключений, функции работают по принципу “все или ничего”.
- Для всех контейнеров ссылки на существующие элементы остаются корректными. Для ассоциативных контейнеров все итераторы, установленные на существующие элементы, остаются корректными. Для неупорядоченных контейнеров итераторы, установленные на существующие элементы, остаются корректными, если не выполняется повторное хеширование (если количество результирующих элементов равно или больше количества сегментов, умноженного на коэффициент максимальной нагрузки).
- Вторая и четвертая версии доступны начиная со стандарта C++11.
- Предусмотрены в множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении.

`iterator контейнер::emplace (args)`

`pair<iterator, bool> контейнер::emplace (args)`

- Вставляет новый элемент, инициализированный списком аргументов *args*, в ассоциативный или неупорядоченный массив.
- Контейнеры, допускающие дубликаты (неупорядоченные), мультимножества и мультиотображения, имеют первую сигнатуру. Они возвращают позицию нового элемента. Начиная со стандарта C++11 вновь вставленные элементы гарантированно размещаются после существующих эквивалентных значений.
- Контейнеры, не допускающие дубликатов (неупорядоченные), множества и отображения, имеют вторую сигнатуру. Если вставка элемента невозможна из-за того, что в контейнере уже существует элемент с таким же значением или ключом, они возвращают позицию существующего элемента и значение `false`. Если вставка возможна, они возвращают позицию нового элемента и значение `true`.
- Если хеш-функции в неупорядоченных контейнерах не генерируют исключений, функции работают по принципу “все или ничего”.
- Для всех контейнеров ссылки на существующие элементы остаются корректными. Для ассоциативных контейнеров все итераторы, установленные на существующие элементы, остаются корректными. Для неупорядоченных контейнеров итераторы,

установленные на существующие элементы, остаются корректными, если не выполняется повторное хеширование (если количество результирующих элементов равно или больше количества сегментов, умноженного на коэффициент максимальной нагрузки).

- Для последовательных контейнеров возможна та же сигнатура, в которой первый аргумент рассматривается как позиция, в которую можно вставить новый элемент (см. раздел 8.7.1).
- Для вставки новой пары “ключ–значение” в (неупорядоченные) отображения и мультиотображения необходимо использовать конструкцию, работающую частями, в виде кортежей (см. раздел 7.8.2).
- Доступны начиная со стандарта C++11.
- Предусмотрены в множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении.

`iterator контейнер::insert (const_iterator pos, const T& value)`

`iterator контейнер::insert (const_iterator pos, T&& value)`

- Вставляет аргумент *value* в позицию итератора *pos*.
- Первая версия копирует аргумент *value*.
- Вторая версия перемещает аргумент *value* в контейнер, так что после выполнения операции значение аргумента *value* становится неопределенным.
- Возвращает позицию нового элемента.
- Если контейнер не допускает дубликатов (множества, неупорядоченные множества, отображения и неупорядоченные отображения) и уже содержит элемент, равный (ключу) *value*, операция не имеет последствий и возвращает позицию существующего элемента.
- Для ассоциативных и неупорядоченных контейнеров позиция используется только как подсказка, откуда можно начинать поиск места для вставки. Если аргумент *value* вставляется прямо в позицию *pos*, то функция имеет амортизированную константную сложность; в противном случае она имеет логарифмическую сложность.
- При работе с векторами эта операция делает итераторы и ссылки на элементы некорректными, если эти элементы предшествуют позиции *pos* и не выполняется перераспределение памяти.
- При работе с деками эта операция делает итераторы некорректными, а если позиция *pos* не совпадает с одним из концов, но некорректными становятся и все ссылки.
- *T* — тип элементов контейнера. Следовательно, для (неупорядоченных) отображений и мультиотображений он представляет собой пару “ключ–значение”.
- Для отображения, мультиотображения, неупорядоченного отображения и неупорядоченного мультимножества соответствующая форма с семантикой перемещения является шаблонной (см. раздел 3.2). Следовательно, аргумент *value* может иметь любой тип, который можно преобразовать в тип значения (пару “ключ–значение”) контейнера. Это позволяет передавать две строки так, чтобы первая из них могла быть преобразована в константную строку (ключ).
- При работе со строками аргумент *value* передается по значению.

- Если при работе с векторами и деками операции копирования и перемещения (конструктор и операция присваивания) элементов не генерируют исключений, то функция работает по принципу “все или ничего”. Если хеш-функция в неупорядоченных контейнерах не генерирует исключений, функция работает по принципу “все или ничего”. Во всех остальных стандартных контейнерах функция работает по принципу “все или ничего”.
- Вторая версия доступна начиная со стандарта C++11. До появления стандарта C++11 использовался тип `iterator`, а не `const_iterator`.
- Предусмотрена в векторе, деке, списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

`iterator контейнер::emplace (const_iterator pos, args)`

- Вставляет новый элемент, инициализированный списком аргументов *args*, в позицию итератора *pos*.
- Возвращает позицию нового элемента.
- При работе с векторами эта операция делает итераторы и ссылки на элементы некорректными, если эти элементы предшествуют позиции *pos* и не выполняется перераспределение памяти.
- При работе с деками эта операция делает итераторы некорректными, а если позиция *pos* не совпадает с одним из концов, но некорректными становятся и все ссылки.
- *T* — тип элементов контейнера.
- Если при работе с векторами и деками операции копирования и перемещения (конструктор и операция присваивания) элементов не генерируют исключений, то функция работает по принципу “все или ничего”. Во всех остальных стандартных контейнерах функция работает по принципу “все или ничего”.
- Для ассоциативных контейнеров возможна та же сигнатура, в которой первый аргумент рассматривается как первый аргумент нового элемента (см. раздел 8.7.1).
- Если хеш-функция в неупорядоченных контейнерах не генерирует исключений, функция работает по принципу “все или ничего”.
- Доступна начиная со стандарта C++11.
- Предусмотрена в векторе, деке и списке.

`iterator контейнер::emplace_hint (const_iterator pos, args)`

- Вставляет новый элемент, инициализированный списком аргументов *args*, в позицию итератора *pos*.
- Возвращает позицию нового элемента.
- Если контейнер не допускает дубликатов (множества, неупорядоченные множества, отображения и неупорядоченные отображения) и уже содержит элемент, равный (ключу) *args*, операция не имеет последствий и возвращает позицию существующего элемента.

- Для ассоциативных и неупорядоченных контейнеров позиция используется только как подсказка, откуда можно начинать поиск места для вставки. Если новый элемент вставляется непосредственно в позицию *pos*, то функция имеет амортизированную константную сложность; в противном случае — логарифмическую сложность.
- *T* — тип элементов контейнера. Следовательно, для (неупорядоченных) отображений и мультиотображений он представляет собой пару “ключ–значение”.
- Если хеш-функция в неупорядоченных контейнерах не генерирует исключений, функция работает по принципу “все или ничего”.
- Доступна начиная со стандарта C++11.
- Предусмотрена в множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении.

```
void контейнер::push_front (const T& value)
```

```
void контейнер::push_front (T&& value)
```

- Вставляют аргумент *value* как новый первый элемент.
- Первая версия копирует аргумент *value*.
- Вторая версия, доступная начиная со стандарта C++11, перемещает аргумент *value* в контейнер, поэтому состояние аргумента *value* после выполнения операции становится неопределенным.
- *T* — тип элементов контейнера.
- Обе версии эквивалентны выражению `insert(begin(), value)`.
- Для деков эта операция делает итераторы на все элементы некорректными. Ссылки на элементы остаются корректными.
- Функции работают по принципу “все или ничего”.¹
- Предусмотрены в деке, списке и последовательном списке.

```
void контейнер::emplace_front (args)
```

- Вставляет новый первый элемент, инициализированный списком аргументов *args*.
- Таким образом, тип элемента должен иметь вызываемый конструктор для списка аргументов *args*.
- Для деков эта операция делает итераторы на все элементы некорректными. Ссылки на элементы остаются корректными.
- Функция работает по принципу “все или ничего”.
- Доступна начиная со стандарта C++11.
- Предусмотрена в деке, списке и последовательном списке.

```
void контейнер::push_back (const T& value)
```

```
void контейнер::push_back (T&& value)
```

- Добавляют аргумент *value* как новый последний элемент.
- Первая версия копирует аргумент *value*.

¹ Для последовательных списков стандарт в настоящее время ничего об этом не говорит, что, вероятно, является его недочетом.

- Вторая версия, доступная начиная со стандарта C++11, перемещает аргумент *value* в контейнер, поэтому состояние аргумента *value* после выполнения операции становится неопределенным.
- *T* — тип элементов контейнера.
- Обе версии эквивалентны выражению `insert(end(), value)`.
- Для векторов эта операция делает итераторы и ссылки на все элементы некорректными, если происходит перераспределение памяти (новое количество элементов превосходит предыдущую емкость).
- Для деков эта операция делает итераторы на все элементы некорректными. Ссылки на элементы остаются корректными.
- Для строк аргумент *value* передается по значению.
- Эти функции работают по принципу “все или ничего” (для векторов и деков это гарантируется тем, что при перераспределении памяти используется копирующий конструктор, а перемещающий конструктор не генерирует исключений).
- Предусмотрены для вектора, дека, списка и строки.

`void контейнер::emplace_back (args)`

- Добавляет новый последний элемент, инициализированный списком аргументов *args*.
- Таким образом, тип элемента должен иметь вызываемый конструктор для списка аргументов *args*.
- Для векторов эта операция делает итераторы и ссылки на все элементы некорректными, если происходит перераспределение памяти (новое количество элементов превосходит предыдущую емкость).
- Для деков эта операция делает итераторы на все элементы некорректными. Ссылки на элементы остаются корректными.
- Эти функции работают по принципу “все или ничего” (для векторов и деков это гарантируется тем, что при перераспределении памяти используется копирующий конструктор, а перемещающий конструктор не генерирует исключений).
- Доступна начиная со стандарта C++11.
- Предусмотрена в векторе, деке и списке.

8.7.2. Вставка нескольких элементов

`void контейнер::insert (список-инициализации)`

- Вставляет копии элементов *списка инициализации* в ассоциативный контейнер.
- Для всех контейнеров ссылки на существующие элементы остаются корректными. Для ассоциативных контейнеров все итераторы, установленные на существующие элементы, остаются корректными. Для неупорядоченных контейнеров итераторы, установленные на существующие элементы, остаются корректными, если не выполняется повторное хеширование (когда количество результирующих элементов равно или больше количества сегментов, умноженного на коэффициент максимального заполнения).

- Доступна начиная со стандарта C++11.
- Предусмотрена в множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении.

`iterator контейнер::insert (const_iterator pos, список-инициализации)`

- Вставляет копии элементов *списка инициализации* в позицию итератора *pos*.
- Возвращает позицию первого вставленного элемента или итератор *pos*, если *список инициализации* пуст.
- При работе с векторами эта операция делает итераторы и ссылки на элементы некорректными, если эти элементы предшествуют позиции *pos* и не выполняется перераспределение памяти.
- При работе с деками эта операция делает итераторы некорректными, а если позиция *pos* не совпадает с одним из концов, но некорректными становятся и все ссылки.
- Для списков функция работает по принципу “все или ничего”.
- Доступен начиная со стандарта C++11.
- Предусмотрена в векторе, деке, списке и строке.

`iterator контейнер::insert (const_iterator pos, size_type num, const T& value)`

- Вставляет *num* копий аргумента *value* в позицию итератора *pos*.
- Возвращает позицию первого вставленного элемента или итератор *pos*, если *num*==0 (до появления стандарта C++11 не возвращала ничего).
- При работе с векторами эта операция делает итераторы и ссылки на элементы некорректными, если эти элементы предшествуют позиции *pos* и не выполняется перераспределение памяти.
- При работе с деками эта операция делает итераторы некорректными, а если позиция *pos* не совпадает с одним из концов, но некорректными становятся и все ссылки.
- *T* — тип элементов контейнера. Следовательно, для отображений и мультиотображений он представляет собой пару “ключ–значение”.
- Для строк аргумент *value* передается по значению.
- Если при работе с векторами и деками операции копирования и перемещения (конструктор и операция присваивания) элементов не генерируют исключений, то функция работает по принципу “все или ничего”. При работе со списками функция работает по принципу “все или ничего”.
- До появления стандарта C++11 использовался тип *iterator*, а не *const_iterator*, а возвращаемое значение имело тип *void*.
- Предусмотрена в векторе, деке, списке и строке.

`void контейнер::insert (InputIterator beg, InputIterator end)`

- Вставляет в ассоциативный контейнер копии всех элементов диапазона *[beg,end)*.
- Эта функция-член является шаблонной (см. раздел 3.2). Следовательно, элементы диапазона-источника могут иметь любой тип, который может быть преобразован в тип элементов контейнера.

- Для всех контейнеров ссылки на существующие элементы остаются корректными. Для ассоциативных контейнеров все итераторы, установленные на существующие элементы, остаются корректными. Для неупорядоченных контейнеров итераторы, установленные на существующие элементы, остаются корректными, если не выполняется повторное хеширование (если количество результирующих элементов равно или больше количества сегментов, умноженного на коэффициент максимального заполнения).
- Если хеш-функции в неупорядоченных контейнерах не генерируют исключений, функции работают по принципу “все или ничего”.
- Предусмотрена в множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении.

`iterator контейнер::insert (const_iterator pos,`
`InputIterator beg, InputIterator end)`

- Вставляет в ассоциативный контейнер копии всех элементов диапазона `[beg, end)` в позицию итератора `pos`.
- Возвращает позицию первого вставленного элемента или итератор `pos`, если `beg==end` (до появления стандарта C++11 функция ничего не возвращала).
- Эта функция-член является шаблонной (см. раздел 3.2). Следовательно, элементы диапазона-источника могут иметь любой тип, который может быть преобразован в тип элементов контейнера.
- При работе с векторами эта операция делает итераторы и ссылки на элементы некорректными, если эти элементы предшествуют позиции `pos` и не выполняется перераспределение памяти.
- При работе с деками эта операция делает итераторы некорректными, а если позиция `pos` не совпадает с одним из концов, но некорректными становятся и все ссылки.
- Для списков функция работает по принципу “все или ничего”.
- До появления стандарта C++11 использовался тип `iterator`, а не `const_iterator`, а возвращаемое значение имело тип `void`.
- Предусмотрена в векторе, деке, списке и строке.

8.7.3. Удаление элементов

`size_type контейнер::erase (const T& value)`

- Удаляет все элементы, эквивалентные аргументу `value`, из ассоциативного или упорядоченного контейнера.
- Возвращает количество удаленных элементов.
- Вызывает деструкторы удаляемых элементов.
- `T` — тип упорядоченных значений:
 - для (неупорядоченных) множеств и мультимножеств — это тип элементов;
 - для (неупорядоченных) отображений и мультиотображений — это тип ключей.

- Функция оставляет корректными итераторы и ссылки на другие элементы.
- Функция может генерировать исключение, если критерий сравнения или хеш-функция генерирует исключения.
- Предусмотрена в множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении.
- Для (последовательных) списков функция `remove()` делает то же самое (см. раздел 8.8.1). Для других контейнеров можно применять алгоритм `remove()` (см. раздел 11.7.1).

`iterator контейнер::erase (const_iterator pos)`

- Удаляет элемент, стоящий в позиции итератора `pos`.
- Возвращает позицию следующего элемента (или `end()`).
- Вызывает деструктор удаляемого элемента.
- Вызывающая сторона должна гарантировать корректность итератора `pos`. Например:

```
coll.erase(coll.end()); // ОШИБКА В непредсказуемые последствия
```

- Для векторов и деков операция может делать некорректными итераторы и ссылки на другие элементы. Для всех остальных контейнеров итераторы и ссылки на другие элементы остаются корректными.
- При работе со списками функция не генерирует исключений. При работе с векторами и деками функция не генерирует исключений при условии, что копирующий/перемещающий конструктор и оператор присваивания не генерируют исключений. При работе с ассоциативными и неупорядоченными контейнерами функция может генерировать исключение, если критерий сравнения или хеш-функция генерирует исключения.
- До появления стандарта C++11 для ассоциативных контейнеров типом возвращаемого значения был `void`, а вместо `const_iterator` использовался тип `iterator`.
- После появления стандарта C++11 вызов функции `erase()` для множеств, элементами которых являются итераторы, может оказаться неоднозначным. По этой причине в настоящее время стандарт C++11 предусматривает перегрузку для обоих вариантов: `erase(iterator)` и `erase(const_iterator)`.
- Предусмотрена в векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

`iterator контейнер::erase (const_iterator beg, const_iterator end)`

- Удаляет деструкторы удаляемых элементов.
- Вызывающая сторона должна гарантировать, что позиции `beg` и `end` определяют корректный диапазон, являющийся частью контейнера.
- Для векторов и деков операция может делать некорректными итераторы и ссылки на другие элементы. Для всех остальных контейнеров итераторы и ссылки на другие элементы остаются корректными.

- При работе со списками функция не генерирует исключений. При работе с векторами и деками функция не генерирует исключений при условии, что копирующий/перемещающий конструктор и оператор присваивания не генерируют исключений. Для ассоциативных и неупорядоченных контейнеров функция может генерировать исключение, если критерий сравнения и функция хеширования генерируют исключения.
- До появления стандарта C++11 для ассоциативных контейнеров типом возвращаемого значения был `void`, а вместо `const_iterator` использовался тип `iterator`.
- Предусмотрен в векторе, деке, списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

`void контейнер::pop_front ()`

- Удаляет первый элемент контейнера.
- Эквивалентна выражению `контейнер.erase(контейнер.begin())` или (для последовательных списков) выражению `контейнер.erase_after(контейнер.before_begin())`
- Если контейнер пуст, последствия непредсказуемы. Следовательно, вызывающая сторона должна гарантировать, что контейнер содержит хотя бы один элемент (`!empty()`).
- Функция не генерирует исключений.
- Итераторы и ссылки на другие элементы остаются корректными.
- Предусмотрена в деке, списке и последовательном списке.

`void контейнер::pop_back ()`

- Удаляет последний элемент контейнера.
- Эквивалентна выражению

`контейнер.erase(prev(контейнер.end()))`

- Если контейнер пуст, последствия непредсказуемы. Следовательно, вызывающая сторона должна гарантировать, что контейнер содержит хотя бы один элемент (`!empty()`).
- Функция не генерирует исключений.
- Итераторы и ссылки на другие элементы остаются корректными.
- Для строк предусмотрена начиная со стандарта C++11.
- Предусмотрена в векторе, деке, списке и строке.

`void контейнер::clear ()`

- Удаляет все элементы (опустошает контейнер).
- Вызывает деструкторы удаляемых элементов.
- Делает некорректными все итераторы и ссылки на элементы контейнера.
- Для векторов, деков и строк функция делает некорректным запредельный итератор, который был возвращен функцией `end()` или `cend()`.
- Функция не генерирует исключений (до появления стандарта C++11 для векторов и деков функция могла генерировать исключения, если копирующий конструктор или операция присваивания генерировал исключения).

- Предусмотрен в векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

8.7.4. Изменение размера

```
void контейнер::resize (size_type num)
void контейнер::resize (size_type num, const T& value)
```

- Изменяют количество элементов до *num*.
- Если `size()` на входе равен *num*, функции не применяются.
- Если *num* на входе больше `size()`, создаются дополнительные элементы, которые добавляются в конец контейнера. Первая версия создает новые элементы, вызывая их конструкторы по умолчанию; вторая версия создает новые элементы как копии аргумента *value*.
- Если *num* на входе меньше `size()`, элементы удаляются с конца контейнера, чтобы размер контейнера стал равным *num*. В этом случае функции вызывают деструкторы удаляемых элементов.
- Для векторов и деков эта операция может делать некорректными итераторы и ссылки на другие элементы. Для всех остальных контейнеров итераторы и ссылки на другие элементы остаются корректными.
- Для векторов и деков эти функции работают по принципу “все или ничего”, при условии, что конструктор или операция присваивания не генерирует исключений. Для списков и последовательных списков функции работают по принципу “все или ничего”.
- До появления стандарта C++11 аргумент *value* передавался по значению.
- Для строк аргумент *value* передается по значению.
- Предусмотрены в векторе, деке, списке, последовательном списке и строке.

8.8. Специальные функции-члены для списков и последовательных списков

8.8.1. Специальные функции-члены для списков (и последовательных списков)

```
void список::remove (const T& value)
void список::remove_if (UnaryPredicate op)
```

- Функция `remove()` удаляет все элементы, имеющие значение, равное *value*.
- Функция `remove_if()` удаляет все элементы, для которых унарный предикат *op(elem)* возвращает `true`.
- Предикат *op* не должен изменять состояние во время вызова функции. Подробности приведены в разделе 10.1.4.
- Обе функции вызывают деструкторы удаляемых элементов.

- Порядок оставшихся элементов остается прежним.
- Функции представляют собой специальные версии алгоритма `remove()`, рассматриваемого в разделе 11.7.1.
- `T` — тип элементов контейнера.
- Дальнейшие подробности изложены в разделе 7.5.2.
- Функции могут генерировать исключения, только если сравнение элементов может генерировать исключения.
- Предусмотрены в списке и последовательном списке.

```
void список::unique ()
```

```
void список::unique (BinaryPredicate op)
```

- Удаляет последующие дубликаты элементов (последовательного) списка, так что значение каждого элемента отличается от значения следующего элемента.
- Первый вариант удаляет все элементы, значения которых равны значениям предыдущих элементов.
- Второй вариант удаляет все элементы, следующие за элементом *e*, для которых бинарный предикат `op(elem, e)` возвращает `true`. Иначе говоря, этот предикат используется для сравнения элемента не с его предшественником, а с предыдущим элементом, который не был удален.
- Предикат *op* не должен изменять состояние во время вызова функции. Подробности приведены в разделе 10.1.4.
- Обе функции вызывают деструкторы удаляемых элементов.
- Функции представляют собой специальные версии алгоритмов `unique()` (см. раздел 11.7.2).
- Функции могут генерировать исключения, только если критерий сравнения элементов может генерировать исключения.
- Предусмотрены в списке и последовательном списке.

```
void список::splice (const_iterator pos, список& source)
```

```
void список::splice (const_iterator pos, список&& source)
```

- Перемещают все элементы из списка *source* в список `*this` и вставляют их перед позицией итератора *pos*.
- После выполнения вызова список *source* остается пустым.
- Если списки *source* и `*this` идентичны, последствия непредсказуемы. Следовательно, вызывающая сторона должна гарантировать, что *source* — другой список. Для перемещения элементов в одном и том же списке следует использовать функции `splice()`, показанные ниже.
- Вызывающая сторона должна гарантировать, что итератор *pos* имеет корректную позицию в списке `*this`; в противном случае последствия непредсказуемы.
- Указатели, итераторы и ссылка на члены списка *source* остаются корректными. Следовательно, после выполнения функций они ссылаются на список *this*.
- Функции не генерируют исключений.

- Вторая версия доступна начиная со стандарта C++11. До появления стандарта C++11 вместо `const_iterator` использовался тип `iterator`.
- Предусмотрены в списке.

```
void список::splice (const_iterator pos, список& source,
const_iterator sourcePos)
void список::splice (const_iterator pos, список&& source,
const_iterator sourcePos)
```

- Перемещает элемент из позиции `sourcePos` списка `source` в список `*this` и вставляет его перед позицией итератора `pos`.
- Списки `source` и `*this` могут быть идентичными. В этом случае элемент перемещается в списке.
- Если список `source` не совпадает со списком `*this`, после выполнения операции он будет содержать на один элемент меньше.
- Вызывающая сторона должна гарантировать, что итератор `pos` установлен на корректную позицию в списке `*this`, итератор `sourcePos` является корректным итератором в списке `source`, а итератор `sourcePos` не совпадает с итератором `source.end()`; в противном случае последствия непредсказуемы.
- Указатели, итераторы и ссылка на члены списка `source` остаются корректными. Следовательно, после выполнения функций они ссылаются на стыковочный элемент в списке `this`.
- Функции не генерируют исключений.
- Вторая версия доступна начиная со стандарта C++11. До появления стандарта C++11 вместо `const_iterator` использовался тип `iterator`.
- Предусмотрены в списке.

```
void список::splice (const_iterator pos, список& source,
const_iterator sourceBeg, const_iterator sourceEnd)
void список::splice (const_iterator pos, список&& source,
const_iterator sourceBeg, const_iterator sourceEnd)
```

- Перемещают элементы диапазона `[sourceBeg,sourceEnd)` списка `source` в список `*this` и вставляют их перед позицией итератора `pos`.
- Списки `source` и `*this` могут быть идентичными. В этом случае итератор `pos` не должен быть частью перемещаемого диапазона, а элементы перемещаются внутри списка.
- Если список `source` не совпадает со списком `*this`, после выполнения операции он будет содержать на несколько элементов меньше.
- Вызывающая сторона должна гарантировать, что итератор `pos` занимает корректную позицию в списке `*this`, а итераторы `sourceBeg` и `sourceEnd` определяют корректный диапазон, являющийся частью списка `source`; в противном случае последствия непредсказуемы.
- Указатели, итераторы и ссылки на другие элементы списка `source` остаются корректными. Следовательно, после выполнения функций они ссылаются на стыковочный элемент в списке `this`.

- Функции не генерируют исключений.
- Вторая версия доступна, начиная со стандарта C++11. До появления стандарта C++11 вместо `const_iterator` использовался тип `iterator`.
- Предусмотрены в списке.

```
void список::sort ()
void список::sort (CompFunc cmpPred)
```

- Упорядочивает элементы.
- Первая версия упорядочивает все элементы с помощью операции `<`.
- Вторая версия упорядочивает все элементы, применяя предикат `cmpPred` для сравнения двух элементов:
- `op (elem1, elem2)`
- Порядок элементов, имеющих одинаковые значения, остается устойчивым, если только не генерируется исключение.
- Эти функции представляют собой специальные версии алгоритмов `sort()` и `stable_sort()` (см. раздел 11.9.1).
- Предусмотрены в списке и последовательном списке.

```
void список::merge (список& source)
void список::merge (список&& source)
void список::merge (список& source, CompFunc cmpPred)
void список::merge (список&& source, CompFunc cmpPred)
```

- Объединяют все элементы (последовательного) списка `source` в списке `*this`.
- После выполнения операции список `source` остается пустым.
- Первые две версии в качестве критерия сортировки используют операцию `<`.
- Последние две версии используют предикат `cmpPred` в качестве необязательного критерия сортировки и для сравнения двух элементов:

```
cmpPred (elem, sourceElem)
```

- Порядок элементов, имеющих одинаковые значения, остается постоянным.
- Если списки `*this` и `source` упорядочены на входе согласно критерию сравнения `<` или `cmpPred`, то результат — (последовательный) список — также упорядочен и эквивалентные элементы списка `*this` предшествуют эквивалентным элементам списка `source`. Строго говоря, стандарт требует, чтобы оба (последовательных) списка на входе были упорядочены. Однако на практике возможно объединение неупорядоченных списков. Впрочем, прежде чем использовать, эту возможность сначала следует проверить.
- Эти функции представляют собой специальную версию алгоритма `merge()` (см. раздел 11.10.2).
- Если критерии сравнения элементов не генерируют исключений, то функции работают по принципу “все или ничего”.
- Предусмотрены в списке и последовательном списке.

`void список::reverse ()`

- Изменяет порядок следования элементов (последовательного) списка.
- Представляет собой специальную версию алгоритма `reverse()` (см. раздел 11.8.1).
- Функция не генерирует исключений.
- Предусмотрена в списке и последовательном списке.

8.8.2. Специальные функции-члены, предназначенные только для последовательных списков

`iterator последовательный_список::before_begin ()`

`const_iterator последовательный_список::before_begin () const`

`const_iterator последовательный_список::cbefore_begin () const`

- Возвращает итератор, установленный на позицию, предшествующую позиции первого элемента.
- Поскольку последовательный список не допускает обратного обхода, эта функция-член позволяет создать позицию для вставки нового первого элемента или удаления существующего первого элемента.
- Предусмотрена в последовательном списке.

`iterator последовательный_список::insert_after (const_iterator pos, const T& value)`

`iterator последовательный_список::insert_after (const_iterator pos, T&& value)`

- Вставляет аргумент `value` непосредственно после позиции итератора `pos`.
- Первая версия копирует аргумент `value`.
- Вторая версия перемещает аргумент `value` в контейнер, так что после выполнения операции состояние аргумента `value` становится неопределенным.
- Возвращает позицию нового элемента.
- Функция работает по принципу “все или ничего”.
- Передача итератора `end()` или `cend()` контейнера в качестве аргумента `pos` приводит к непредсказуемым последствиям.
- Предусмотрена в последовательном списке.

`iterator последовательный_список::emplace_after (const_iterator pos, args)`

- Вставляет новый элемент, инициализированный списком аргументов `args`, непосредственно после позиции итератора `pos`.
- Возвращает позицию нового элемента.
- Функция работает по принципу “все или ничего”.²
- Передача итератора `end()` или `cend()` контейнера в качестве аргумента `pos` приводит к непредсказуемым последствиям.
- Предусмотрена в последовательном списке.

²В настоящее время стандарт ничего не говорит об этом, что, вероятно, является его недочетом.

iterator *последовательный_список*::**insert_after** (const_iterator *pos*, size_type *num*, const T& *value*)

- Вставляет *num* копий аргумента *value* непосредственно после позиции итератора *pos*.
- Возвращает позицию последнего вставленного элемента или итератор *pos*, если *num*==0.
- Функция работает по принципу “все или ничего”.
- Передача итераторов *end()* или *cend()* контейнера в качестве аргумента *pos* приводит к непредсказуемым последствиям.
- Предусмотрена в последовательном списке.

iterator *последовательный_список*::**insert_after** (const_iterator *pos*, *список-инициализации*)

- Вставляет копии элементов *списка инициализации* непосредственно после позиции итератора *pos*.
- Возвращает позицию последнего вставленного элемента или итератор *pos*, если *список инициализации* пуст.
- Функция работает по принципу “все или ничего”.
- Передача итераторов *end()* или *cend()* контейнера в качестве аргумента *pos* приводит к непредсказуемым последствиям.
- Предусмотрена в последовательном списке.

iterator *последовательный_список*::**insert_after** (const_iterator *pos*, InputIterator *beg*, InputIterator *end*)

- Вставляет копии всех элементов диапазона [*beg*,*end*) непосредственно после итератора *pos*.
- Возвращает позицию последнего вставленного элемента или итератор *pos*, если *beg*==*end*.
- Эта функция-член является шаблонной (см. раздел 3.2). Следовательно, элементы диапазона-источника могут иметь любой тип, который может быть преобразован в тип элементов контейнера.
- Функция работает по принципу “все или ничего”.
- Передача итераторов *end()* или *cend()* контейнера в качестве аргумента *pos* приводит к непредсказуемым последствиям.
- Предусмотрена в последовательном списке.

iterator *последовательный_список*::**erase_after** (const_iterator *pos*)

- Удаляет элемент, стоящий непосредственно после позиции итератора *pos*.
- Возвращает позицию следующего элемента (или *end()*).
- Вызывает деструктор удаляемого элемента.
- Итераторы и ссылки на другие элементы остаются корректными.
- Вызывающая сторона должна гарантировать корректность итератора *pos*, исключая обращение к позиции *end()* и позиции, предшествующей *end()*.
- Функция не генерирует исключений.

- Передача итераторов `end()` или `cend()` контейнера в качестве аргумента *pos* приводит к непредсказуемым последствиям.
- Предусмотрена в последовательном списке.

`void последовательный_список::erase_after (const_iterator beg, const_iterator end)`

- Удаляет элементы диапазона (*beg*, *end*). Обратите внимание, что это *не* полуоткрытый диапазон, потому что из него исключены позиции *beg* и *end*. Например:

```
coll.erase(coll.before_begin(), coll.end()); // OK: стирает все элементы
```

- Возвращает позицию *end*.
- Вызывает деструкторы удаляемых элементов.
- Вызывающая сторона должна гарантировать, что позиции *beg* и *end* определяют корректный диапазон, являющийся частью контейнера.
- Функция не генерирует исключений.
- Итераторы и ссылки на другие элементы остаются корректными.
- Предусмотрена в последовательном списке.

`void последовательный_список::splice_after (const_iterator pos, последовательный_список& source)`

`void последовательный_список::splice_after (const_iterator pos, последовательный_список&& source)`

- Перемещают все элементы из списка *source* в список **this* и вставляют их в позицию, непосредственно после итератора *pos*.
- После выполнения вызова список *source* становится пустым.
- Если списки *source* и **this* идентичны, последствия непредсказуемы. Следовательно, вызывающая сторона должна гарантировать, что *source* — другой список. Для перемещения элементов в одном и том же списке следует использовать одну из версий функции `splice_after()`.
- Вызывающая сторона должна гарантировать, что итератор *pos* имеет корректную позицию в списке **this*; в противном случае последствия непредсказуемы.
- Указатели, итераторы и ссылка на члены списка *source* остаются корректными. Следовательно, после выполнения функций они ссылаются на стыковочный элемент в списке *this*.
- Функции не генерируют исключений.
- Передача итератора `end()` или `cend()` контейнера в качестве аргумента *pos* приводит к непредсказуемым последствиям.
- Предусмотрены в последовательном списке.

`void последовательный_список::splice_after (const_iterator pos, последовательный_список& source, const_iterator sourcePos)`

`void последовательный_список::splice_after (const_iterator pos, последовательный_список&& source, const_iterator sourcePos)`

- Перемещает элемент из позиции *sourcePos* списка *source* в список **this* и вставляет его непосредственно после итератора *pos*.

- Списки *source* и **this* могут быть идентичными. В этом случае элемент перемещается внутри списка.
- Если список *source* не совпадает со списком **this*, после выполнения операции он будет содержать на один элемент меньше.
- Вызывающая сторона должна гарантировать, что итератор *pos* установлен на корректную позицию в списке **this*, итератор *sourcePos* является корректным итератором в списке *source*, а итератор *sourcePos* не совпадает с итератором *source*. *end()*; в противном случае последствия непредсказуемы.
- Указатели, итераторы и ссылка на члены списка *source* остаются корректными. Следовательно, после выполнения функций они принадлежат списку *this*.
- Передача итератора *end()* или *cend()* контейнера в качестве аргумента *pos* приводит к непредсказуемым последствиям.
- Предусмотрены в последовательном списке по стандарту C++11.

```
void последовательный_список::splice_after (const_iterator pos,
последовательный_список& source, const_iterator sourceBeg,
const_iterator sourceEnd)
void последовательный_список::splice_after (const_iterator pos,
последовательный_список&& source, const_iterator sourceBeg,
const_iterator sourceEnd)
```

- Перемещают элементы диапазона (*sourceBeg,sourceEnd*) списка *source* в список **this* и вставляют их в позицию итератора *pos*. Обратите внимание, что это не полуоткрытый диапазон, потому что из него исключены позиции *SourceBeg* и *SourceEnd*. Например, следующий вызов перемещает все элементы списка *coll2* в начало списка *coll*.

```
coll.splice_after(coll.before_begin(), coll2,
coll2.before_begin(), coll2.end());
```

- Списки *source* и **this* могут быть идентичными. В этом случае итератор *pos* не должен быть частью перемещаемого диапазона, а элементы перемещаются внутри списка.
- Если список *source* не совпадает со списком **this*, после выполнения операции он будет содержать на несколько элементов меньше.
- Вызывающая сторона должна гарантировать, что итератор *pos* занимает корректную позицию в списке **this*, а итераторы *sourceBeg* и *sourceEnd* определяют корректный диапазон, являющийся частью списка *source*; в противном случае последствия непредсказуемы.
- Указатели, итераторы и ссылки на другие элементы списка *source* остаются корректными. Следовательно, после выполнения операции они принадлежат списку *this*.
- Функции не генерируют исключений.
- Передача итераторов *end()* или *cend()* контейнера в качестве аргумента *pos* приводит к непредсказуемым последствиям.
- Предусмотрены в последовательном списке.

8.9. Интерфейсы стратегий

8.9.1. Немодифицирующие вспомогательные функции

`size_type контейнер::capacity () const`

- Возвращает количество элементов контейнера, которые он может содержать без перераспределения памяти.
- Предусмотрена в векторе и строке.

`value_compare контейнер::value_comp () const`

- Возвращает объект, используемый в качестве критерия сравнения значений в ассоциативных массивах.
- В множествах и мультимножествах эквивалентна `key_comp()`.
- В отображениях и мультиотображениях представляет собой вспомогательный класс для критерия сравнения, сравнивающего только ключи из пар “ключ–значение”.
- Предусмотрена в множестве, мультимножестве, отображении и мультиотображении.

`key_compare контейнер::key_comp () const`

- Возвращает критерий сравнения для ассоциативных контейнеров.
- Предусмотрена в множестве, мультимножестве, отображении и мультиотображении.

`key_equal контейнер::key_eq () const`

- Возвращает критерий сравнения для неупорядоченных контейнеров.
- Предусмотрена в неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении и неупорядоченном мультиотображении.

`hasher контейнер::hash_function () const`

- Возвращает хеш-функцию для неупорядоченных контейнеров.
- Предусмотрена в неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении и неупорядоченном мультиотображении.

`float контейнер::load_factor () const`

- Возвращает текущее среднее количество элементов в сегменте неупорядоченного контейнера.
- Предусмотрена в неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении и неупорядоченном мультиотображении.

`float контейнер::max_load_factor () const`

- Возвращает коэффициент максимального заполнения неупорядоченного контейнера. Контейнер автоматически выполняет повторное хеширование (увеличивает количество сегментов при необходимости), чтобы коэффициент нагрузки был меньше или равен этому числу.
- По умолчанию это число равно 1.0 и обычно должно уточняться (см. раздел 7.9.2).
- Предусмотрена в неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении и неупорядоченном мультиотображении.

8.9.2. Модифицирующие вспомогательные функции

`void контейнер::reserve (size_type num)`

- Резервирует внутреннюю память как минимум для *num* элементов.
- Для векторов эта функция может только увеличивать емкость. Следовательно, если *num* меньше или равно текущей емкости, функция никаких действий не выполняет. Для уменьшения емкости векторов используется функция `shrink_to_fit()` (см. пример в разделе 7.3.1).
- Для неупорядоченных контейнеров
 - этот вызов эквивалентен вызову `rehash(ceil(num/max_load_factor))` (функция `ceil()` возвращает число, округленное в сторону увеличения);
 - эта операция делает некорректными итераторы, изменяет порядок следования элементов, а также изменяет сегменты, в которых расположены элементы; указатели и ссылки на элементы остаются корректными.
- Для строк аргумент *num* является необязательным (по умолчанию: 0), и вызов представляет собой несвязанный запрос на уменьшение размера, если число *num* меньше реальной емкости.
- Эта функция может делать некорректными итераторы, а также (для векторов и строк) ссылки и указатели на элементы. Однако гарантируется, что повторное выделение памяти после вызова функции `reserve()` не произойдет, пока после очередной вставки размер не превысит число *num*. Таким образом, функция `reserve()` может повышать быстродействие и способствовать сохранению корректности ссылок, указателей и итераторов (см. раздел 7.3.1).
- Генерирует исключение `length_error` (см. раздел 4.3.1), если *num* > `max_size()`, или соответствующее исключение, если выделение памяти закончилось неудачей.
- Доступна для неупорядоченных контейнеров начиная со стандарта C++11.
- Предусмотрена в векторе, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

`void контейнер::shrink_to_fit ()`

- Уменьшает объем внутренней памяти до объема, требуемого для хранения в точности имеющегося количества элементов.
- Этот вызов представляет собой несвязывающий запрос, т.е. реализации могут игнорировать его, чтобы обеспечить специализированную оптимизацию. Таким образом, нет гарантии, что после выполнения этой операции выражение `capacity() == size()` вернет значение `true`.
- Эта операция может делать некорректными ссылки, указатели и итераторы, установленные на элементы.
- Доступна начиная со стандарта C++11. Пример уменьшения емкости векторов до появления стандарта C++11 см. в разделе 7.3.1.
- Предусмотрена в векторе, деке и строке.

`void контейнер::rehash (size_type bnum)`

- Изменяет количество сегментов неупорядоченного контейнера как минимум до *bnum*.
- Эта операция делает некорректными итераторы, изменяет порядок элементов, и изменяет сегменты, в которых размещены элементы. Указатели и ссылки на элементы остаются корректными.
- Если генерируется исключение, которое отличается от исключения, генерируемого контейнерной хеш-функцией или функцией сравнения, операция не выполняет никаких действий.
- Для неупорядоченных мультимножеств и мультиотображений повторное хеширование сохраняет относительный порядок следования эквивалентных элементов.
- Предусмотрена в неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

`void контейнер::max_load_factor (float loadFactor)`

- Устанавливает коэффициент максимального заполнения неупорядоченного контейнера равным *loadFactor*.
- Аргумент *loadFactor* интерпретируется как подсказка, поэтому реализации могут уточнять это значение по своему усмотрению.
- Эта операция может вызвать повторное хеширование, делающее некорректными итераторы, изменяющее порядок элементов и сегменты, в которых размещены элементы. Указатели и ссылки на элементы остаются корректными.
- Предусмотрена в неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

8.9.3. Сегментный интерфейс для неупорядоченных контейнеров

`size_type контейнер::bucket_count () const`

- Возвращает текущее количество сегментов неупорядоченного контейнера.
- Предусмотрена в неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

`size_type контейнер::max_bucket_count () const`

- Возвращает максимально возможное количество сегментов неупорядоченного контейнера.
- Предусмотрена в неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

`size_type контейнер::bucket (const key_type key) const`

- Возвращает индекс сегмента, в котором можно найти элементы с ключом, эквивалентным аргументу *key*, если таковые существуют.
- Возвращаемое значение лежит в диапазоне `[0, bucket_count())`.

- Если функция `bucket_count()` возвращает нуль, возвращаемое значение функции `bucket()` является неопределенным.
- Предусмотрена в неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

`size_type контейнер::bucket_size (size_type bucketIdx) const`

- Возвращает количество элементов в сегменте с индексом `bucketIdx`.
- Если индекс `bucketIdx` не является корректным значением из диапазона `[0, bucket_count())`, последствия непредсказуемы.
- Предусмотрена в неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

`local_iterator контейнер::begin (size_type bucketIdx)`

`const_local_iterator контейнер::begin (size_type bucketIdx) const`

`const_local_iterator контейнер::cbegin (size_type bucketIdx) const`

- Все три версии возвращают итератор, установленный в начало всех элементов (позицию первого элемента) сегмента с индексом `bucketIdx`.
- Если сегмент пустой, то этот вызов эквивалентен вызовам `контейнер::end (bucketIdx)` или `контейнер::cend (bucketIdx)` соответственно.
- Если индекс `bucketIdx` не является корректным значением из диапазона `[0, bucket_count())`, последствия непредсказуемы.
- Предусмотрена в неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении.

`local_iterator контейнер::end (size_type bucketIdx)`

`const_local_iterator контейнер::end (size_type bucketIdx) const`

`const_local_iterator контейнер::cend (size_type bucketIdx) const`

- Все три версии возвращают итератор, установленный в конец всех элементов (позицию после последнего элемента) сегмента с индексом `bucketIdx`.
- Если сегмент пустой, то этот вызов эквивалентен вызовам `контейнер::begin (bucketIdx)` или `контейнер::cbegin (bucketIdx)` соответственно.
- Если индекс `bucketIdx` не является корректным значением из диапазона `[0, bucket_count())`, последствия непредсказуемы.
- Предусмотрены в неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении.

8.10. Функции для выделения памяти

Все контейнеры STL можно использовать совместно со специальной моделью памяти, определенной распределителем памяти (см. главу 19). В этом разделе описываются члены, поддерживающие работу распределителя.

8.10.1. Основные члены распределителя памяти

контейнер : `allocator_type`

- Тип распределителя.
- Предусмотрен в векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке

`allocator_type` *контейнер* : `get_allocator () const`

- Возвращает модель памяти контейнера.
- Предусмотрен в векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

8.10.2. Конструкторы для необязательных параметров распределителя памяти

`explicit` *контейнер* : `контейнер (const Allocator& alloc)`

- Создает новый пустой контейнер, использующий модель памяти *alloc*.
- Предусмотрен в векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

контейнер : `контейнер (const CompFunc& cmpPred, const Allocator& alloc)`

- Создает новый пустой контейнер с предикатом *cmpPred*, используемым как критерий сортировки, и моделью памяти *alloc*.
- Критерий сортировки должен определять *строгое слабое упорядочение* (см. раздел 7.7).
- Предусмотрен в множестве, мультимножестве, отображении и мультиотображении.

контейнер : `контейнер (size_type bnum, const Hasher& hasher, const KeyEqual& eqPred, const Allocator& alloc)`

- Создает новый пустой контейнер, имеющий не менее *bnum* сегментов, функцию хеширования *hasher*, предикат *eqPred*, используемый как критерий для идентификации одинаковых значений, и модель памяти *alloc*.
- Предусмотрен в неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении и неупорядоченном мультиотображении.

контейнер : `контейнер (список-инициализации, const Allocator& alloc)`

- Создает новый контейнер, использующий модель памяти *alloc* и инициализированный элементами *списка инициализации*.

- Доступен начиная со стандарта C++11.
- Предусмотрен в векторе, деке, списке, последовательном списке и строке.

`контейнер::контейнер` (*список-инициализации*, `const CompFunc& cmpPred`, `const Allocator& alloc`)

- Создает новый пустой контейнер с предикатом *cmpPred*, используемым как критерий сортировки, и моделью памяти *alloc*, инициализированной элементами *списка инициализации*.
- Критерий сортировки должен определять *строгое слабое упорядочение* (см. раздел 7.7).
- Доступен начиная со стандарта C++11.
- Предусмотрен в множестве, мультимножестве, отображении и мультиотображении.

`контейнер::контейнер` (*список-инициализации*, `size_type bnum`, `const Hasher& hasher`, `const KeyEqual& eqPred`, `const Allocator& alloc`)

- Создает новый пустой контейнер, имеющий не менее *bnum* сегментов, функцию хеширования *hasher*, предикат *eqPred*, используемый как критерий для идентификации одинаковых значений, модель памяти *alloc* и инициализированный элементами *списка инициализации*.
- Предусмотрена в неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении и неупорядоченном мультиотображении.

`контейнер::контейнер` (`const контейнер& c`, `const Allocator& alloc`)
`контейнер::контейнер` (`контейнер&& c`, `const Allocator& alloc`)

- Создает новый контейнер, использующий модель памяти *alloc* и инициализированный скопированными/перемещенными элементами существующего контейнера *c*.
- Вызывает копирующий/перемещающий конструктор каждого элемента контейнера *c*.
- После вызова второй версии контейнер *c* остается корректным, но имеет неопределенное значение.
- Доступен начиная со стандарта C++11.
- Предусмотрен в векторе, деке, списке, последовательном списке, множестве, мультимножестве, отображении, мультиотображении, неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении, неупорядоченном мультиотображении, строке.

`контейнер::контейнер` (`size_type num`, `const T& value`, `const Allocator& alloc`)

- Создает контейнер, содержащий *num* элементов и использующий модель памяти *alloc*.
- Элементы создаются как копии аргумента *value*.
- *T* — тип элементов контейнера. Для строк аргумент *value* передается по значению.
- Предусмотрен в векторе, деке, списке, последовательном списке и строке.

`контейнер::контейнер` (`InputIterator beg`, `InputIterator end`, `const Allocator& alloc`)

- Создает контейнер, инициализированный всеми элементами диапазона *[beg,end)* и использующий модель памяти *alloc*.

- Эта функция-член является шаблонной (см. раздел 3.2). Следовательно, элементы диапазона-источника могут иметь любой тип, который может быть преобразован в тип элементов контейнера.
- Предусмотрен в векторе, деке, списке, последовательном списке и строке.

`контейнер::контейнер` (InputIterator *beg*, InputIterator *end*,
const CompFunc& *cmpPred*, const Allocator& *alloc*)

- Создает контейнер, имеющий критерий сортировки *cmpPred* и инициализированный всеми элементами диапазона [*beg*,*end*) и использующий модель памяти *alloc*.
- Эта функция-член является шаблонной (см. раздел 3.2). Следовательно, элементы диапазона-источника могут иметь любой тип, который может быть преобразован в тип элементов контейнера.
- Критерий сортировки должен определять *строгое слабое упорядочение* (см. раздел 7.7).
- Предусмотрен в множестве, мультимножестве, отображении и мультиотображении.

`контейнер::контейнер` (InputIterator *beg*, InputIterator *end*,
size_type *bnum*, const Hasher& *hasher*, const KeyEqual& *eqPred*,
const Allocator& *alloc*)

- Создает контейнер, имеющий не менее *bnum* сегментов, функцию хеширования *hasher*, критерий эквивалентности *eqPred*, использующий модель памяти *alloc* и инициализированный всеми элементами диапазона [*beg*,*end*).
- Предусмотрена в неупорядоченном множестве, неупорядоченном мультимножестве, неупорядоченном отображении и неупорядоченном мультиотображении.

